

تقرير المرحلة الثانية – الطلاب والتحقق

المبسّط من الهوية (Phase 2 – Students)

(Simplified KYC &

اسم المرحلة ورقمها

المرحلة الثانية: الطلاب والتحقق المبسّط من الهوية (KYC) – Phase 2

الهدف من المرحلة

السماح للطلاب بالتسجيل ببيانات بروفایل بسيطة (Lean Profile)، وتقديم بيانات تحقق هوية مبسطة (KYC): رقم البطاقة الشخصية، صورة البطاقة، وصورة الطالب. مع بناء البنية التحتية اللازمة لاحقًا حتى يستطيع أصحاب المباني الاطلاع على بيانات الطلاب المرتبطين بمبانيهم فقط — وهي النقطة التي ظهر فيها انحراف مهم عن الخطة الأصلية، موضح بالتفصيل في قسم "أي انحراف عن الخطة الأصلية" أدناه.

الملفات والمجلدات التي اتعملت فعلياً

1. وحدة الطلاب (Students Module)

src/modules/students/

— student.model.js

→ قاعدة البيانات (البروفایل البسيط فقط)

— student.validation.js

→ المطلوبة والاختيارية (zod) قواعد التحقق

— student.repository.js

→ طبقة الوصول لقاعدة البيانات

— student.service.js

→ منطق التسجيل وتحديث البروفایل

└─ student.controller.js → HTTP معالجات طلبات
└─ student.routes.js → (register / me / update) تعريف المسارات

2. وحدة ال KYC

src/modules/kyc/
└─ kyc.model.js → نموذج سجل التحقق (منفصل عن بروفایل الطالب)
└─ kyc.repository.js → طبقة الوصول لقاعدة البيانات
└─ kyc.service.js → ق الإنشاء وإعادة التقديم وتحديث حالة التحقق
└─ kyc.controller.js → validation لا يوجد ملف) معالجات الطلبات
└─ kyc.routes.js → (me / resubmit / status) تعريف المسارات

3. البنية التحتية المشتركة (جديدة هذه المرحلة)

src/shared/utils/
└─ file-storage.adapter.js → (storeFile / getFile) واجهة تخزين الملفات

4. ملفات معدّلة

src/modules/auth/auth.service.js → getUserById() إضافة
src/app.entry.js → /api/students و /api/ ربط مسارات

5. ملفات الاختبار

tests/integration/students-kyc.test.js → (في القسم الخاص بالاختبارات)

شرح تفصيلي لكل خطوة اتفدت

الخطوة 1: قرار تخزين الملفات (منفذ حسب القرار المعتمد مسبقًا)

تم تنفيذ `file-storage.adapter.js` خلف واجهة موحدة (`storeFile` , `deleteFile` , `getFileName` , `Resolved Before Implementation` في `phase-2-students-kyc.md` . التنفيذ الحالي هو محاكاة محلية (**mock**): يكتب الملفات في مجلد مؤقت على السيرفر (`os.tmpdir()/sakanify-mock-storage` , أو `MOCK_STORAGE_DIR` عبر متغير بيئة)، ويرجع `reference` بشكل مطابق تمامًا لشكل الـ `key` الحقيقي (`kyc/<uuid>.<ext>`) و `url` شبيه بروابط `S3/R2` الموقعة (تنتهي صلاحيتها). عند توفير بيانات اعتماد `Cloudflare R2` لاحقًا، التغيير سيقصر على هذا الملف فقط — بدون أي تعديل في `student.service` أو `kyc.service`.

قرار تقني إضافي غير مذكور صراحةً في المستند الأصلي: تم إضافة فحص حقيقي لمحتوى الملف (`magic bytes`) للتأكد أن الملف صورة `JPEG/PNG/WEBP` فعليًا، وليس فقط الاعتماد على حقل `mimetype` المرسل من العميل (وهو حقل قابل للتزوير بسهولة). هذا تطبيق مباشر لقاعدة `CLAUDE.md` البند 3.8. تم اختبار هذا السلوك صراحة (انظر قسم الاختبارات).

الخطوة 2: نموذج الطالب (`student.model.js`)

تم تنفيذ الحقول المحددة حرفيًا وبدون أي إضافة: `name` , `phone` (منسوخ من حساب المصادقة عند التسجيل), `email` (اختياري), `age` (اختياري), `profile_photo` (اختياري, مرجع فقط وليس بيانات ثنائية خام), `college` , `academic_year` , `university_id` (اختياري), `smoking_preference` . لم تتم إضافة أي حقل غير مذكور (لا صورة شخصية للتحقق البيومتري, لا فصيلة دم, لا نمط نوم, لا مستوى هدوء مفضل) — التزامًا صارمًا بقسم 5.1 من `CLAUDE.md`.

فهرسة (Indexes): `user` (فريد), `college` , `phone` — معرّفة مباشرة مع النموذج حسب قسم 4.1.

الخطوة 3: نموذج KYC (kyc.model.js) كمجموعة منفصلة

تم بناء `kyc.model.js` كمجموعة (collection) مستقلة تمامًا عن بروفایل الطالب، ترتبط به عبر `student` (`ObjectId`)، بحيث يمكن تتبع حالة التحقق ومراجعتها بشكل مستقل — كما هو مطلوب في الخطوة 2 من المستند الأصلي.

الحقول الحساسة (`national_id_photo`, `national_id_number`) معرّفة بـ `select: false` بحيث لا تظهر أبدًا في أي استعلام افتراضي — طبقة حماية إضافية غير مذكورة صراحة في المستند لكنها تطبيق مباشر لروح قسم 3 من CLAUDE.md (تقليل التعرض غير الضروري للبيانات الحساسة).

الخطوة 4: التحقق من صحة البيانات (student.validation.js)

تم استخدام مكتبة `zod` (موجودة بالفعل ضمن `dependencies` المشروع) لتعريف الحقول المطلوبة (`name`, `college`, `academic_year`, `smoking_preference`) والاختيارية (`age`, `email`, `university_id`). رقم البطاقة القومية يُتحقق منه بشكل منفصل (نمط 14 رقمًا) داخل `student.controller.js` و `kyc.controller.js` مباشرة، وليس عبر ملف `validation` منفصل لموديول الـ KYC — وذلك مطابقةً حرفيًا لهيكل المجلدات المحدد في `phase-2-students-kyc.md`، الذي لا يذكر ملف `kyc.validation.js` أصلًا.

الخطوة 5: حالة التحقق (verification_status)

الحقل `verification_status` يبدأ بقيمة `pending` تلقائيًا عند إنشاء سجل الـ KYC، مع endpoint مخصص لتغييره إلى `verified` أو `rejected`. تم اتخاذ قرار بأن هذا الـ endpoint **مقتصر على Super-Admin فقط في هذه المرحلة** (وليس صاحب المبنى) — والسبب موضح بالتفصيل في قسم الانحراف عن الخطة أدناه.

كل تغيير في حالة التحقق يسجّل عليه `reviewed_by` (المستخدم الذي نُقِّد التغيير) و `reviewed_at` (التوقيت) — تطبيقًا لقسم 3.9/5.3 من CLAUDE.md الخاص بتتبع أي تغيير على بيانات حساسة لفاعل وزمن محددين.

الخطوة 6: endpoint التسجيل (بروفایل + KYC معًا)

POST /api/students/register — endpoint محمي (يتطلب توكن طالب صادر من نظام المصادقة في المرحلة الأولى)، يستقبل بيانات البروفايل + رقم البطاقة القومية + صورتين (multipart/form-data)، وينشئ سجل الطالب وسجل ال KYC الأولي في عملية واحدة متسلسلة (الخطوة السادسة في المستند الأصلي).

الخطوة 7: نقاط تحديث منفصلة

- PATCH /api/students/me — تحديث الحقول العامة للبروفايل، متاح في أي وقت.
- POST /api/kyc/me/resubmit — إعادة تقديم بيانات ال KYC، **مسموح فقط عندما تكون حالة التحقق الحالية rejected** — أي محاولة إعادة تقديم أثناء pending أو verified تُرفض (Conflict 409). هذا تطبيق حرفي للخطوة السابعة في المستند الأصلي.

الخطوة 8: endpoint اطلاع صاحب المبنى على بيانات الطالب — لم يُنفَّذ في هذه المرحلة

راجع قسم "أي انحراف عن الخطة الأصلية" أدناه — هذا قرار تم اتخاذه بالتشاور المسبق مع صاحب المشروع (Amr) قبل كتابة أي كود.

أي قرارات تقنية اتأخذت أثناء التنفيذ

1. إضافة `authService.getUserById()`: موديول الطلاب يحتاج رقم هاتف المستخدم عند التسجيل، لكن ال JWT الصادر من المرحلة الأولى لا يحمل رقم الهاتف ضمن ال payload (فقط `userId`, `role`, `ownerId`). بدلاً من الوصول المباشر لموديول `User` من داخل موديول الطلاب (وهو ما يخالف قسم 7.2 من CLAUDE.md بخصوص عزل الموديولات)، تمت إضافة دالة خدمة بسيطة `getUserById()` في `auth.service.js` يستدعيها `student.service.js`.

2. فحص محتوى الملفات الفعلي (magic bytes) بدلاً من الاعتماد فقط على `mimetype` — موضح في الخطوة 1 أعلاه.

3. **تقييد تحديث حالة التحقق (verify/reject) على Super-Admin فقط** بدلاً من صاحب المبنى — قرار أمني لتفادي فجوة عزل بيانات حقيقية، موضح بالتفصيل في القسم التالي.

4. **سجل KYC واحد فقط لكل طالب** (وليس سجل جديد لكل محاولة تقديم) — إعادة التقديم تُحدّث نفس السجل بدلاً من إنشاء سجل جديد، حفاظاً على تاريخ تحقق واحد قابل للتتبع بدلاً من عدة سجلات متفرقة.

الاختبارات التي اتعملت والنتائج

تم إنشاء `tests/integration/students-kyc.test.js` ويغطي:

- تسجيل طالب جديد وإنشاء بروفایل + سجل KYC أولي (`pending`) معًا.
- رفض التسجيل المكرر لنفس الحساب (409).
- رفض التسجيل عند نقص الحقول المطلوبة (422).
- رفض رقم بطاقة قومية غير مكوّن من 14 رقمًا (422).
- **رفض ملف حقيقي غير صورة حتى لو كان اسمه وامتداده يوحيان بأنه صورة (400)** — يتحقق من أن فحص الـ `magic bytes` يعمل فعليًا، وليس فقط `mimetype`.
- استرجاع البروفايل الخاص بالطالب (`GET /me`) مع حالة الـ KYC، بدون ظهور رقم البطاقة القومية أو صورتها في الاستجابة أبدًا (تحقق من `select: false`).
- تحديث الحقول العامة للبروفايل دون التأثير على بيانات الـ KYC.
- رفض إعادة تقديم KYC أثناء حالة `pending` (409).
- قبول إعادة التقديم بعد الرفض (`pending` → `rejected`) بنجاح.
- موافقة Super-Admin على KYC (`verified`) مع تسجيل `reviewed_by` / `reviewed_at`.
- رفض قيمة حالة غير صالحة عند التحديث (400).
- **اختبارات حدود الأدوار (Role Guards):** رفض توكن صاحب مبنى على `endpoint` تسجيل الطلاب (403)، رفض توكن طالب على `endpoint` تحديث حالة الـ KYC (403)، رفض توكن صاحب مبنى على نفس الـ `endpoint` (403)، رفض الوصول بدون توكن (401).

• الاختبار الحرج البديل المذكور صراحة في تعليمات المرحلة: بدلاً من اختبار "صاحب مبنى A لا يستطيع رؤية بيانات طالب غير مرتبط به" (وهو اختبار غير قابل للكتابة بمعناه الحقيقي في هذه المرحلة — راجع القسم التالي)، تم كتابة اختبار صريح باسم واضح يوثق ويتحقق من أنه لا يوجد أصلاً أي endpoint يسمح لصاحب مبنى بالوصول لبيانات أي طالب في هذه المرحلة (الاستجابة 404 وليست 403 — لأن المسار غير موجود بالأساس، وليس لأن الصلاحية رُفضت).

نتيجة التشغيل الفعلي: تم التحقق من كل الاختبارات أعلاه فعلياً عبر GitHub Actions (سيرفر حقيقي مع MongoDB ك service container) — **النتيجة: نجاح كامل (Success)**، تشغيل رقم 4 على `main branch`، `commit 0372ab5` ("Phase 2"). رابط التشغيل: <https://github.com/vdivcompany-cmd/Sakanify-backend/actions/runs/30663307013>

قبل ذلك، تم التحقق محلياً فقط من خلو كل الملفات الجديدة والمعدّلة من أخطاء syntax (`node --check` على كل ملف — نجح الجميع)، لأن بيئة الـ `sandbox` لا تستطيع تحميل ثنائي MongoDB اللازم لـ `mongodb-memory-server` بسبب قيود الشبكة (نفس القيد الموثّق سابقاً في ذاكرة المشروع بخصوص الوصول لـ MongoDB Atlas). هذا القيد لا يؤثر على GitHub Actions لأنه يشغّل MongoDB حقيقياً كخدمة منفصلة، وهو ما تم الاعتماد عليه كمصدر التحقق النهائي والوحيد الموثوق به.

أي انحراف عن الخطة الأصلية

الانحراف الرئيسي: عدم بناء endpoint اطلاع صاحب المبنى على بيانات الطالب (الخطوة 8 و9 في المستند الأصلي)

النص الأصلي في `phase-2-students-kyc.md` يطلب:

Build the owner-facing endpoint to view a student's profile"
KYC data — restricted (via the Phase 1 ownership-scoping +
helper) to only students connected to that owner's
".buildings through an active or pending request/rental

Test explicitly: Owner A must not be able to view KYC data"

".for a student with no relationship to Owner A's buildings

المشكلة: عند فحص حالة المستودع الفعلية قبل كتابة أي كود، تبين أن موديلات **Buildings (المرحلة 3)** و **Rentals/Requests (المرحلة 4)** غير مُنفَّذة بعد — كل ملفاتها موجودة ك placeholders فارغة (0 سطر). العلاقة التي من المفترض أن يُبنى عليها التحقق من الملكية ("طالب مرتبط بمبنى صاحب معيّن عبر طلب أو إيجار نشط/معلّق") **غير موجودة أصلاً في قاعدة البيانات حتى الآن.**

بناء endpoint لاطلاع لصاحب المبنى بدون آلية ربط حقيقية كان سيعني أحد أمرين، وكلاهما مرفوض حسب قواعد CLAUDE.md:

1. بناء endpoint مفتوح بدون تحقق فعلي من الملكية — مخالفة مباشرة لقسم 3.3 و 3.11 (الأمان الافتراضي الأكثر تقييداً).
2. بناء آلية ربط مؤقتة ومُختَرَعَة (guessed) بدون تحديد من صاحب المشروع — مخالفة مباشرة لقسم 7.5 (أي قرار تصميمي غامض يُرْفَع للمراجعة بدلاً من التخمين).

تم طرح السؤال على صاحب المشروع (Amr) قبل كتابة أي كود، وكان القرار المعتمد: تأجيل بناء هذا الـ endpoint بالكامل إلى المرحلة الرابعة (**Booking Engine**)، حيث ستكون علاقة Rentals/Requests موجودة فعلياً ليُبنى عليها التحقق من الملكية بشكل صحيح وآمن. تم توثيق هذا القرار في كود المصدر (تعليقات في `student.routes.js` و `kyc.service.js`) وفي اختبار صريح يؤكد عدم وجود أي مسار وصول غير مُصرَّح به حالياً.

الأثر على "الاختبار الحرج" المطلوب صراحة في تعليمات المرحلة: بما أن الـ endpoint نفسه غير موجود، لا يمكن كتابة اختبار "صاحب مبنى A لا يرى بيانات طالب غير مرتبط به" بمعناه الحقيقي (لا يوجد شيء ليُختبر رفضه). تم استبداله باختبار صريح وواضح الاسم يوثّق ويتحقق من عدم وجود أي endpoint من هذا النوع أصلاً، مع توصية بإعادة كتابة الاختبار الأصلي بمعناه الكامل فور تنفيذ المرحلة الرابعة.

انحرافات ثانوية

- **تقييد تحديث حالة الـ KYC (verify/reject) على Super-Admin فقط بدلاً من صاحب المبنى:** نفس السبب الجذري أعلاه — منح صاحب المبنى صلاحية تغيير حالة تحقق أي طالب في النظام (وليس طلابه فقط) كان سيمثل فجوة عزل بيانات حقيقية، لأن لا توجد بعد وسيلة لتحديد "طلاب هذا

المبنى تحديداً". القرار: صاحب المبنى سيحصل على هذه الصلاحية (مقيدة بطلابه فقط) في المرحلة الرابعة مع باقي endpoint الاطلاع.

- **سجل تدقيق (Audit Log) مركزي غير مُفعّل بعد:** قسم 3.9 من CLAUDE.md يطلب تسجيل أي تغيير على بيانات حساسة (KYC ضمنها) في سجل تدقيق مركزي. موديول `src/modules/audit` موجود ك `placeholder` فارغ فقط (لم يُنَّ في أي مرحلة سابقة). بدلاً من بناء نظام تدقيق كامل خارج نطاق هذه المرحلة (وهو ما لم يُطلب صراحة في قائمة ملفات المرحلة الثانية)، تم الاكتفاء بحقلي `reviewed_at / reviewed_by` مباشرة على سجل الـ KYC نفسه كحل مؤقت يحافظ على قابلية التتبع. **توصية:** بناء موديول `audit` الفعلي ينبغي أن يسبق المرحلة الخامسة (المدفوعات)، لأن قسم 5.3 يعتبر سجل التدقيق "مصدر الحقيقة" في أي نزاع على المدفوعات.

الحالة النهائية للمرحلة

الحالة: مكتملة بالكامل ومُتحقّق منها فعلياً (Fully Complete & Verified).

تم في البداية تأجيل الـ `push` بسبب ملف `T:\sakanify\.git\index.lock` عالق على جهاز المستخدم (خطأ "Operation not permitted" عند محاولة حذفه)، وهو ما وثّقه إصدار سابق من هذا التقرير. تم لاحقاً حل المشكلة على جهاز المستخدم (إغلاق برنامج الـ Git الذي كان محتفظاً بالقفل وحذف الملف يدوياً)، وتم بعدها بنجاح:

1. الـ `commit` والـ `push` إلى `origin/main` — `0372ab5 commit` بعنوان "Phase 2".
2. تشغيل **GitHub Actions** تلقائياً على الـ "Sakanify workflow" — `push — workflow` "Backend — Tests"، التشغيل رقم #4.
3. النتيجة: نجاح كامل (Success) — كل الاختبارات المذكورة في القسم أعلاه اجتازت الفحص على سيرفر حقيقي مع MongoDB كخدمة فعلية (service container)، وليس محاكاة.

رابط التشغيل الفعلي على **GitHub Actions**: <https://github.com/vdivcompany-cmd/Sakanify-backend/actions/runs/30663307013>

بهذا تُغلق المرحلة الثانية رسميًا، وأصبح موديول الطلاب والـ KYC جاهزًا كأساس تُبنى عليه المرحلة الثالثة (Buildings/Apartments/Beds)، بما في ذلك التوصية المؤتقة أعلاه ببناء موديول **audit** الحقيقي — وهي التوصية التي تم تبنيها فعليًا وسحبها للأمام في مواصفة المرحلة الثالثة.